Episode Alerts

Mobile Application Project Report

Project Title:	Episode Alerts Mobile App
Technology Stack:	React Native, Expo, TypeScript
Prepared By:	Nandu Krishna M 25CSE2015
Prepared For:	Object Oriented Software Engineering Lab
Date:	April 16, 2026

Contents

Chapter 1

Introduction

1.1 Project Introduction

Episode Alerts is a mobile application designed to help users discover television shows, organize personal watchlists, and stay informed about upcoming episodes. The project is implemented using React Native, Expo, and TypeScript, with TMDB as the primary source for show metadata and content discovery.

1.2 Objective

The objective of this project is to provide a practical, user-friendly companion app for TV tracking that combines discovery, watchlist management, reminders, and preferences in a single workflow. The implementation focuses on responsive interaction, modular service design, and reliable behavior in both online and cache-assisted scenarios.

1.3 Problem Statement

TV viewers often rely on scattered sources and manual reminders to keep track of shows, episode releases, and personal watch progress. This leads to missed episodes, inconsistent watchlist maintenance, and a fragmented user experience across discovery, details, and tracking tasks. The problem addressed by Episode Alerts is to provide a single, reliable mobile workflow that unifies show discovery, detailed show information, watchlist management, reminder scheduling, and preference handling, while remaining usable even when network availability is limited through cache-assisted behavior and optional cloud synchronization.

Chapter 2

Background (Related Work)

2.1 Background

TV tracking is fragmented when users have to move between search, release calendars, watchlists, and reminders. Episode Alerts addresses that by combining show discovery, show details, watchlist management, notifications, and cached fallback behavior in one mobile app.

2.2 Related Work

A recent App Store release, CineSync Tracker, follows the same core direction: a unified movie and TV watchlist, a release calendar, search/discovery, and a minimalist interface.

The original Expo post by the app author, this post, presents CineSync Tracker as a free minimalist alternative built with Expo and designed around utility.

The follow-up comment exchange, this thread, confirms TMDB as the data source and notes the free-tier limit of about 40 requests every 10 seconds. That supports a cache-first design and fallback handling in Episode Alerts.

2.3 Project Positioning

Episode Alerts is TV-first, cache-aware, and focused on practical tracking rather than general catalog browsing.

Chapter 3

SRS

3.1 Introduction

3.1.1 Purpose

This software requirements specification describes the functional and non-functional requirements for Episode Alerts, a cross-platform mobile app that helps users discover television shows, manage a personal watchlist, and stay updated on upcoming episodes. It is intended for developers, testers, reviewers, and future maintainers who need a clear understanding of what the system should do and the limits it must work within.

3.1.2 Scope

Episode Alerts is built with React Native, Expo, and TypeScript, and it uses the TMDB API as its primary content source. The app supports show discovery, detailed show and season views, watchlist management, episode notifications, user preferences, offline-friendly caching, and optional cloud synchronization. It is designed as a companion app rather than a streaming platform.

3.1.3 Definitions, Acronyms, and Abbreviations

API Application Programming Interface.

AsyncStorage
Local key-value storage used to persist app data on the device.

DFD Data Flow Diagram.

Expo The framework and toolchain used to build and run the app.

Firebase
Cloud platform used for optional authentication and data synchronization.

FR Functional Requirement.

SRS Software Requirements Specification.

TMDB The Movie Database, the external service used for show data.

UI User Interface.

3.1.4 References

A consolidated list of the external documents and sources used in this report is provided in the References chapter at the end of the document.

3.1.5 Overview

This SRS is organized into three main parts. The introduction explains the purpose and scope of the product, the overall description summarizes the product context and main characteristics, and the specific requirements section covers the external interfaces, functional behavior, performance expectations, constraints, and quality attributes.

3.2 Overall Description

3.2.1 Product Perspective

Episode Alerts is a single-user mobile app with a modular, service-oriented architecture. Navigation is handled through Expo Router, with tab-based screens for Home, Search, Watchlist, and Settings, while modal and detail screens support deeper navigation flows. The app depends on TMDB for content data, AsyncStorage for local persistence, and optional Firebase services for cloud synchronization.

3.2.2 Product Function

At a high level, the product provides the following functions:

- Discover popular, top-rated, and currently airing television shows.
- Search for shows by title and view detailed information.
- Display season and episode data, including next and last episode details.
- Let users add, remove, and manage shows in a watchlist.
- Notify users about upcoming episodes and release events.
- Store user preferences, analytics settings, and cached content.
- Synchronize selected user data to the cloud when enabled.

3.2.3 User Characteristics

The primary users are everyday TV viewers who want a simple way to follow multiple shows without manually tracking air dates. Most users are expected to have little or no technical background, so the app needs to stay easy to navigate, visually clear, and responsive. More advanced users may rely more heavily on search, filters, sorting, and settings.

3.2.4 General Constraints

The system is constrained by the Expo and React Native runtime, the availability of the TMDB API, and platform permission requirements for notifications and, where enabled, cloud features. The app must run on Android, iOS, and web with shared code, and it must keep sensitive values such as API keys outside the source tree. Offline behavior is limited to locally cached data and stored user content.

3.2.5 Assumptions and Dependencies

The system assumes that TMDB remains available and that the user has network access whenever fresh show data is needed. It also assumes that device permissions can be granted for notifications and that local storage is available for caching and preferences. Optional Firebase features depend on valid configuration and a signed-in user account.

3.3 Specific Requirements

3.3.1 External Interface Requirements

3.3.1.1 User Interfaces

The user interface shall provide a clean tab-based experience with Home, Search, Watchlist, and Settings screens. Detail pages shall present show metadata, season information, episode lists, and countdown information in a readable layout. The interface shall support light, dark, and system themes, show loading and error states, and remain usable across different screen sizes.

3.3.1.2 Hardware Interfaces

The application shall support touch input on mobile devices and pointer or keyboard input on web. It shall use device storage for local persistence and, when notifications are enabled, the operating system notification subsystem. If calendar or device-specific

sharing features are added in future updates, the app shall request the relevant platform permissions before access.

3.3.1.3 Software Interfaces

The application shall integrate with the TMDB API for all show, season, and episode data. It shall use AsyncStorage for local caching and user preference storage, Expo Notifications for reminders, Expo Network for connectivity awareness, and optional Firebase authentication and Firestore for cloud synchronization. The app shall also work with React Navigation and Expo Router for screen transitions.

3.3.1.4 Communication Interfaces

All external data exchange shall use secure HTTPS requests. The app shall communicate with TMDB through REST endpoints and shall use Firebase network services only when cloud sync is configured and the user is signed in. Notification interactions shall be handled through the platform notification response channel so tapping a reminder can open the related show.

3.3.2 Functional Requirements

3.3.2.1 Model 1: Discovery and Search

FR-1.1 The system shall load and display popular, top-rated, and currently airing TV shows from TMDB.

FR-1.2 The system shall allow the user to search for TV shows by title or keyword.

FR-1.3 The system shall present a featured show on the home screen and allow the user to refresh the content.

FR-1.4 The system shall use cached data when live data is unavailable so the user can still view previously loaded content.

3.3.2.2 Model 2: Show Details and Seasons

FR-2.1 The system shall display show metadata such as overview, rating, status, genres, and network information.

FR-2.2 The system shall display season information and the episodes available for each season.

FR-2.3 The system shall show next episode and last episode details when the information is available from TMDB.

FR-2.4 The system shall present episode countdown and episode details in a dedicated view.

3.3.2.3 Model 3: Watchlist Management

FR-3.1 The system shall allow the user to add a show to the watchlist from discovery or detail screens.

FR-3.2 The system shall allow the user to remove a show from the watchlist.

FR-3.3 The system shall persist the watchlist locally so it is available after the app restarts.

FR-3.4 The system shall preserve watch progress and watch history for shows in the watchlist.

FR-3.5 The system shall support sorting and filtering of watchlist content where applicable in the user interface.

3.3.2.4 Model 4: Notifications and Preferences

FR-4.1 The system shall let the user enable or disable episode notifications.

FR-4.2 The system shall request notification permissions before scheduling reminders.

FR-4.3 The system shall schedule notifications for upcoming episodes when notifications are enabled.

FR-4.4 The system shall allow the user to change theme, analytics, and notification preferences.

FR-4.5 The system shall open the relevant show when the user taps a notification.

3.3.2.5 Model 5: Synchronization and Analytics

FR-5.1 The system shall support optional cloud synchronization of watchlist and preference data.

FR-5.2 The system shall synchronize local changes to the cloud when the user is signed in and cloud sync is available.

FR-5.3 The system shall record screen views and selected user actions when analytics are enabled.

FR-5.4 The system shall respect the user's analytics preference and stop collecting events when analytics are disabled.

3.3.3 Performance Requirements

The application should remain responsive while data is loaded or synchronized in the background. Cached content should appear quickly when available, and common actions such as adding or removing a show from the watchlist should finish without noticeable delay. Notification synchronization and cloud synchronization should not block the main user interface.

3.3.4 Design Constraints

The implementation shall remain within the React Native, Expo, and TypeScript stack already used by the project. The system shall rely on TMDB as the primary content source and shall not require direct changes to third-party services. Local persistence shall continue to use AsyncStorage, and cloud synchronization shall remain optional so the app can still work without Firebase configuration.

3.3.5 Attributes

The system shall prioritize usability, reliability, portability, maintainability, and security. The interface should be easy to understand for non-technical users, data should persist consistently across sessions, and the codebase should stay modular so new features can be added without major restructuring.

3.3.6 Other Attributes

The application should support offline use where cached data is available, preserve user privacy by keeping analytics opt-in, and remain practical for everyday mobile use. Future work may extend support for more detailed personalization, additional calendar integration, or broader cloud features without changing the core product goals.

Chapter 4

Design

4.1 UML

4.1.1 Sequence Diagram

Figure 4.1 illustrates the UML sequence diagram representing the interaction between different components of the system, including user actions, application services, caching mechanisms, and external APIs.

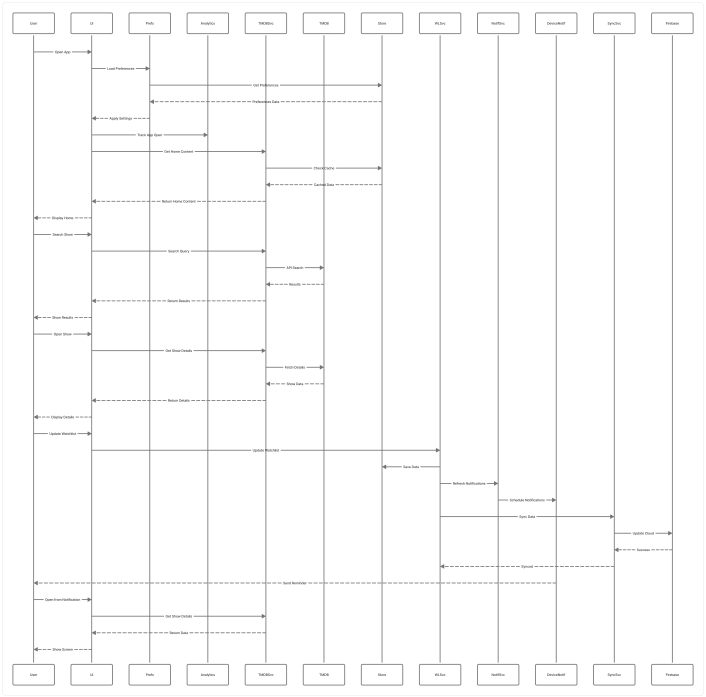


Figure 4.1: UML Sequence Diagram of the System Workflow

4.2 DFD

Episode Alerts is built around a few core data paths: show discovery and search through TMDB, watchlist and preference storage on the device, episode reminders through the notification layer, and optional cloud sync through Firebase.

4.2.1 Level 0 Context DFD

This level shows Episode Alerts as a single system interacting with the user and its external services.

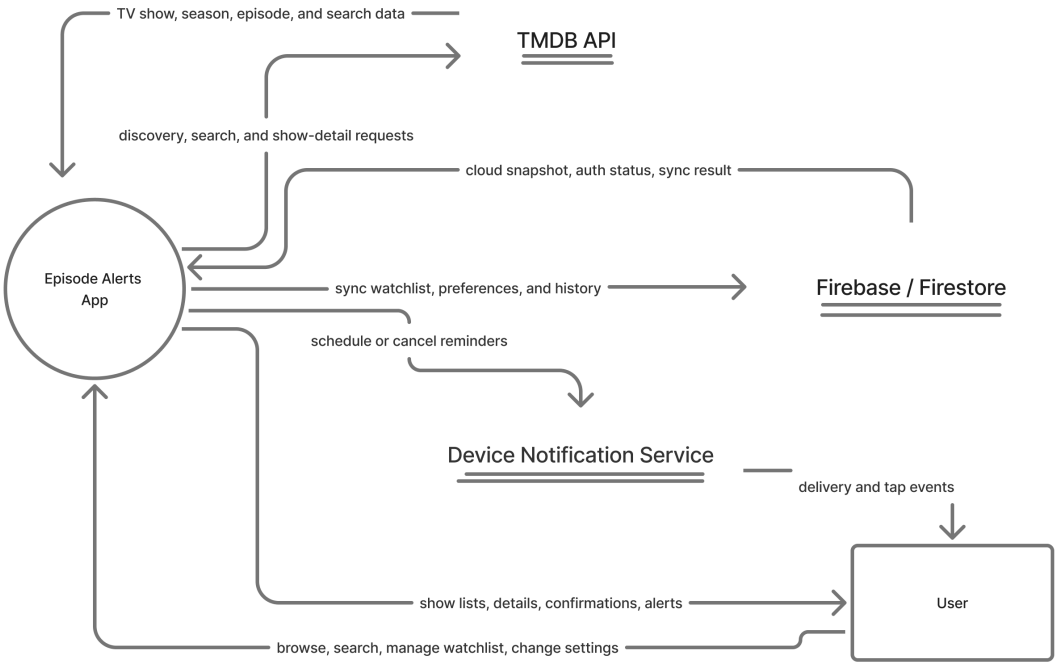


Figure 4.2: Level 0 context DFD for Episode Alerts.

4.2.2 Level 1 Logical DFD

This level breaks the app into its main logical processes and the data stores they use.

4.2.3 Level 2 DFD: Content Discovery and Search

This level decomposes the content discovery path because it is the main entry point into the app and the clearest place to show TMDBService, cache hydration, request normalization, and stale-data fallback. It is intentionally more detailed than level 1.

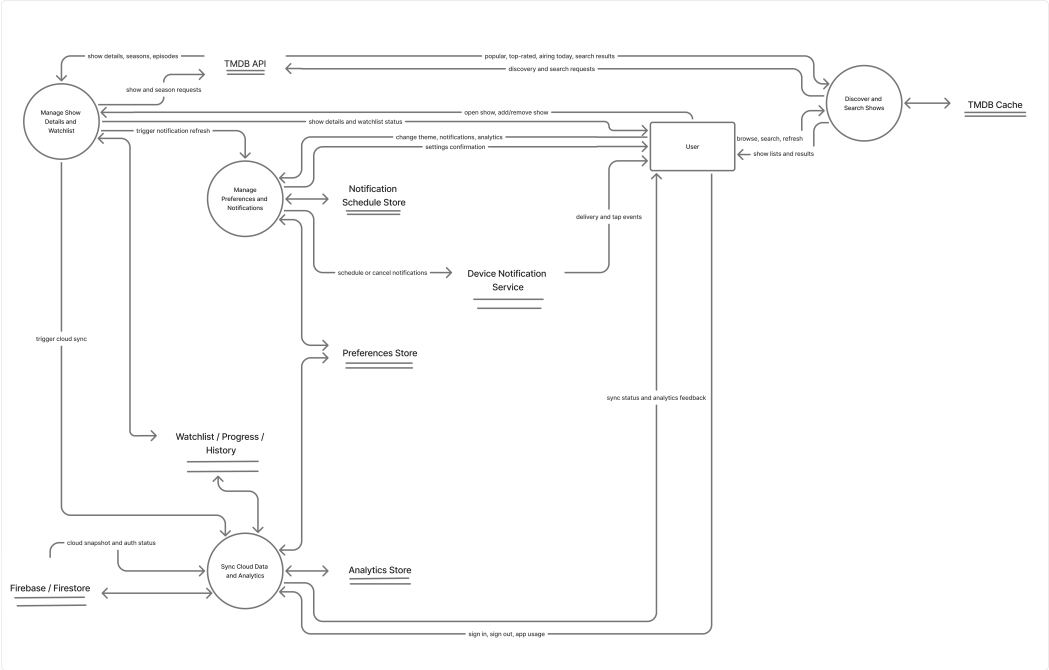


Figure 4.3: Level 1 logical DFD showing the primary processes and data stores.

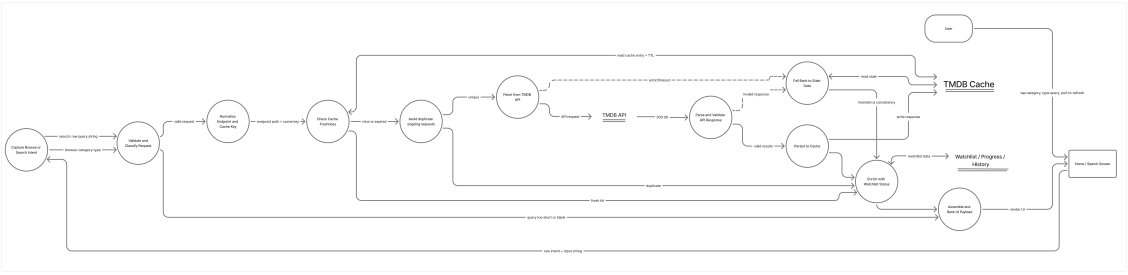


Figure 4.4: Level 2 DFD for content discovery and search, including cache and stale-data fallback paths.

4.3 Use Cases

System

Episode Alerts is a mobile companion app for discovering and tracking television shows.

Assumptions

- TMDB is available when fresh show data is requested, or cached data is already stored on the device.
- The user has granted notification permissions if episode reminders are enabled.
- Firebase is configured only when optional cloud synchronization is being used.
- Local device storage is available for watchlist data, preferences, and cached content.

The Main Use Cases Are

- Discover TV Shows
- Search for TV Shows
- View Show Details
- Manage Watchlist
- Configure Preferences and Notifications
- Sync Data to Cloud

1. Use Case 1: Discover TV Shows

1.1 Primary Actor: User
1.2 Precondition: The app is open and the Home screen is available.
1.3 Scenario: The user browses the Home screen to explore featured and trending content.

1.3.1 Main Success Scenario

1. The user opens the Home screen.
2. The system loads featured, airing today, popular, and top-rated shows from TMDB or from cached data.
3. The system displays the show sections in a scrollable layout.
4. The user taps a show card.
5. The system opens the show details screen for the selected show.

1.3.2 Exception Scenario

- **2a.** If step 2 fails because the network is unavailable, the system shows stale cached data and a stale-data indicator.
- **4a.** If step 4 cannot be completed because the selected item is unavailable, the system stays on the Home screen.

2. Use Case 2: Search for TV Shows

2.1 Primary Actor: User
2.2 Precondition: The Search screen is available and the user can enter text.
2.3 Scenario: The user enters a query to find a specific TV show.

2.3.1 Main Success Scenario

1. The user opens the Search screen.
2. The user types a show title or keyword.
3. The system waits for a valid query and sends the request to TMDB.
4. The system displays the matching search results.
5. The user selects one of the results.
6. The system opens the show details screen for that result.

2.3.2 Exception Scenario

- **3a.** If step 3 fails because the request is too short, the system clears the results and waits for a valid search term.
- **3b.** If the network request fails, the system shows an error message and allows the user to retry.
- **4a.** If no results are returned in step 4, the system shows an empty state instead of a result list.

3. Use Case 3: View Show Details

3.1 Primary Actor: User
3.2 Precondition: The user has selected a show from Home, Search, or Watchlist.
3.3 Scenario: The user opens a show to review detailed information and explore seasons or episodes.

3.3.1 Main Success Scenario

1. The user opens a show details screen.

2. The system loads show metadata such as overview, rating, status, genres, and network information.
3. The system displays next episode information, last episode information, and available seasons.
4. The user opens a season to view its episode list.
5. The user returns to the show details screen or continues to another related action, such as adding the show to the watchlist.

3.3.2 Exception Scenario

- **2a.** If step 2 fails, the system falls back to cached details when available.
- **3a.** If a show does not have a next episode yet, the countdown area is hidden instead of showing invalid information.
- **4a.** If season data cannot be loaded, the system shows an error or empty state for that section only.

4. Use Case 4: Manage Watchlist

4.1 Primary Actor: User
4.2 Precondition: The user is viewing a show card, show details screen, or the Watchlist screen.
4.3 Scenario: The user adds, removes, or organizes shows in the watchlist.

4.3.1 Main Success Scenario

1. The user taps the watchlist action for a show.
2. The system saves the show locally in the watchlist store.
3. The system updates the watchlist screen and related counts immediately.
4. The system schedules release notifications when notifications are enabled.
5. The user later removes the show or clears the watchlist if needed.

4.3.2 Exception Scenario

- **1a.** If step 1 targets a show that is already in the watchlist, the system leaves the list unchanged and returns a duplicate-state response.
- **2a.** If step 2 fails because storage is unavailable, the system shows an error and keeps the previous watchlist state.
- **5a.** If the user clears the watchlist and later chooses undo, the system restores the last saved snapshot when available.

5. Use Case 5: Configure Preferences and Notifications

5.1 Primary Actor: User
5.2 Precondition: The Settings screen is available.
5.3 Scenario: The user changes app preferences such as theme, analytics, and notification behavior.

5.3.1 Main Success Scenario

1. The user opens the Settings screen.
2. The user changes the theme, analytics preference, or notification setting.
3. The system saves the preference locally and updates the app behavior.
4. If notifications are enabled, the system requests permission and schedules upcoming episode reminders.
5. The updated setting remains active the next time the app is opened.

5.3.2 Exception Scenario

- **2a.** If a preference update cannot be saved, the system shows an error and retains the previous value.
- **3a.** If analytics are disabled, the system stops recording usage events immediately.
- **4a.** If notification permission is denied in step 4, the system keeps notifications disabled and does not schedule reminders.

6. Use Case 6: Sync Data to Cloud

6.1 Primary Actor: Signed-in User
6.2 Precondition: Firebase is configured and the user is signed in.
6.3 Scenario: The system synchronizes watchlist data and user preferences with the cloud.

6.3.1 Main Success Scenario

1. The user signs in or triggers a synchronization event.
2. The system builds a payload from the watchlist, watch progress, watch history, and selected preferences.
3. The system uploads the payload to Firestore.
4. The system stores the last successful sync time.
5. The system repeats synchronization automatically when local data changes or when the app resumes.

6.3.2 Exception Scenario

- **1a.** If step 1 occurs while the device is offline, the system skips the upload and retries later.
- **3a.** If Firebase is not configured or the user is not signed in, the system keeps all data local.
- **5a.** If the cloud request fails, the system preserves local data and logs the sync failure for later retry.

Chapter 5

Code

5.1 Implementation Overview

The codebase is centered around a small set of services that support the app's main flow: fetching TV data, managing the user's watchlist, scheduling notifications, and persisting preferences. For this report, the two most important modules are TMDBService and WatchlistService because they drive the core data flow and the user's personalized tracking experience.

5.1.1 TMDBService

TMDBService is the app's main data layer for TV show content. It communicates with TMDB, normalizes show and episode data, and adds caching so the app can keep working even when the network is slow or unavailable. This service is central to the Home, Search, and Show Details screens because almost all show information comes through it.

```
import { TMDB_CONFIG, API_KEY } from '../../constants/Config';
import AsyncStorage from '@react-native-async-storage/async-
    storage';
import { reportError } from '@/app/utils/errorHandling';

export interface TVShow {
  id: number;
  name: string;
  overview: string;
  poster_path: string;
  backdrop_path: string;
  vote_average: number;
  first_air_date: string;
  next_episode_to_air?: Episode;
  last_episode_to_air?: Episode;
```

```
  number_of_seasons: number;
  number_of_episodes?: number;
  status: string;
  genres: Genre[];
  networks: Network[];
  created_by: Creator[];
  seasons?: Season[];
  addedAt?: number;
}

export interface Episode {
  id: number;
  name: string;
  overview: string;
  still_path: string;
  air_date: string;
  episode_number: number;
  season_number: number;
  vote_average: number;
  runtime?: number;
}

export interface Season {
  id: number;
  name: string;
  overview: string;
  poster_path: string;
  air_date: string;
  season_number: number;
  episode_count: number;
  episodes?: Episode[];
}

export interface Genre {
  id: number;
  name: string;
}

export interface Network {
  id: number;
  name: string;
```

```
  logo_path: string;
}

export interface Creator {
  id: number;
  name: string;
  profile_path: string;
}

export interface CastMember {
  id: number;
  name: string;
  profile_path: string | null;
  character?: string;
  order?: number;
}

interface APIResponse<T> {
  page?: number;
  results: T[];
  total_pages?: number;
  total_results?: number;
}

interface CachedResponseEntry {
  data: unknown;
  expiresAt: number;
  updatedAt: number;
}

interface TVShowCreditsResponse {
  cast: CastMember[];
}

const API_CACHE_STORAGE_KEY = '@EpisodeAlerts:tmdbApiCache';
const DEFAULT_CACHE_TTL_MS = 15 * 60 * 1000;
const MAX_CACHE_ENTRIES = 200;
const PERSIST_DEBOUNCE_MS = 400;
const MAX_STALE_RETENTION_MS = 7 * 24 * 60 * 60 * 1000;

class TMDBService {
```

```
97    private static instance: TMDBService;
98    private baseURL: string;
99    private headers: HeadersInit;
100   private memoryCache = new Map<string, CachedResponseEntry>();
101   private inFlightRequests = new Map<string, Promise<unknown>>();
102   private isCacheHydrated = false;
103   private hydratePromise: Promise<void> | null = null;
104   private persistTimer: ReturnType<typeof setTimeout> | null =
        null;
105   private staleFallbackUsed = false;
106   private lastCachedDataUpdatedAt: number | null = null;
107
108   private constructor() {
109     this.baseURL = TMDB_CONFIG.BASE_URL;
110     this.headers = {
111       'Authorization': `Bearer ${API_KEY}`,
112       'Content-Type': 'application/json',
113     };
114   }
115
116   public static getInstance(): TMDBService {
117     if (!TMDBService.instance) {
118       TMDBService.instance = new TMDBService();
119     }
120     return TMDBService.instance;
121   }
122
123   private normalizeEndpoint(endpoint: string): string {
124     const [path, queryString] = endpoint.split('?');
125     if (!queryString) {
126       return path;
127     }
128
129     const params = new URLSearchParams(queryString);
130     const sortedEntries = Array.from(params.entries()).sort((a, b)
          => {
131       if (a[0] === b[0]) {
132         return a[1].localeCompare(b[1]);
133       }
134       return a[0].localeCompare(b[0]);
135     });
```

```
136
137     const normalizedParams = new URLSearchParams();
138     sortedEntries.forEach(([key, value]) => {
139       normalizedParams.append(key, value);
140     });
141
142     return `${path}?${normalizedParams.toString()}`;
143   }
144
145   private getCacheTTL(endpoint: string): number {
146     const normalized = this.normalizeEndpoint(endpoint);
147
148     if (normalized.startsWith('/search/')) {
149       return 5 * 60 * 1000;
150     }
151
152     if (normalized.startsWith('/tv/airing_today')) {
153       return 10 * 60 * 1000;
154     }
155
156     if (normalized.startsWith('/tv/popular') || normalized.
          startsWith('/tv/top_rated')) {
157       return 30 * 60 * 1000;
158     }
159
160     if (/^\/tv\/\d+\/season\/\d+/.test(normalized)) {
161       return 6 * 60 * 60 * 1000;
162     }
163
164     if (/^\/tv\/\d+/.test(normalized)) {
165       return 60 * 60 * 1000;
166     }
167
168     return DEFAULT_CACHE_TTL_MS;
169   }
170
171   private buildCacheKey(endpoint: string): string {
172     return this.normalizeEndpoint(endpoint);
173   }
174
175   private async ensureCacheHydrated(): Promise<void> {
```

```
176     if (this.isCacheHydrated) {
177       return;
178     }
179
180     if (this.hydratePromise) {
181       await this.hydratePromise;
182       return;
183     }
184
185     this.hydratePromise = (async () => {
186       try {
187         const raw = await AsyncStorage.getItem(
              API_CACHE_STORAGE_KEY);
188         if (!raw) {
189           this.isCacheHydrated = true;
190           return;
191         }
192
193         const parsed = JSON.parse(raw) as Record<string,
              CachedResponseEntry>;
194         const now = Date.now();
195         Object.entries(parsed).forEach(([key, entry]) => {
196           if (!entry || typeof entry.expiresAt !== 'number') {
197             return;
198           }
199
200           const isFresh = entry.expiresAt > now;
201           const isWithinStaleRetention = now - entry.expiresAt <=
                MAX_STALE_RETENTION_MS;
202
203           if (isFresh || isWithinStaleRetention) {
204             this.memoryCache.set(key, entry);
205           }
206         });
207         this.isCacheHydrated = true;
208       } catch (error) {
209         reportError('TMDBService.ensureCacheHydrated', error, {
210           fallbackMessage: 'Failed to hydrate cached API data.',
211           trackAnalytics: false,
212         });
213         this.isCacheHydrated = true;
```

```
214       } finally {
215         this.hydratePromise = null;
216       }
217     })();
218
219     await this.hydratePromise;
220   }
221
222   private getCachedResponse<T>(cacheKey: string): { data: T | null
       ; isStale: boolean; updatedAt: number | null } {
223     const entry = this.memoryCache.get(cacheKey);
224     if (!entry) {
225       return {
226         data: null,
227         isStale: false,
228         updatedAt: null,
229       };
230     }
231
232     if (entry.expiresAt <= Date.now()) {
233       return {
234         data: entry.data as T,
235         isStale: true,
236         updatedAt: entry.updatedAt,
237       };
238     }
239
240     return {
241       data: entry.data as T,
242       isStale: false,
243       updatedAt: entry.updatedAt,
244     };
245   }
246
247   private trimCacheIfNeeded(): void {
248     if (this.memoryCache.size <= MAX_CACHE_ENTRIES) {
249       return;
250     }
251
252     const oldestEntries = Array.from(this.memoryCache.entries())
253       .sort((a, b) => a[1].updatedAt - b[1].updatedAt)
```

```
254       .slice(0, this.memoryCache.size - MAX_CACHE_ENTRIES);
255
256     oldestEntries.forEach(([key]) => {
257       this.memoryCache.delete(key);
258     });
259   }
260
261   private setCachedResponse(cacheKey: string, data: unknown, ttlMs
        : number): void {
262     const now = Date.now();
263     this.memoryCache.set(cacheKey, {
264       data,
265       expiresAt: now + ttlMs,
266       updatedAt: now,
267     });
268
269     this.trimCacheIfNeeded();
270     this.scheduleCachePersist();
271   }
272
273   private scheduleCachePersist(): void {
274     if (this.persistTimer) {
275       clearTimeout(this.persistTimer);
276     }
277
278     this.persistTimer = setTimeout(() => {
279       this.persistTimer = null;
280       this.persistCache().catch((error) => {
281         reportError('TMDBService.persistCache', error, {
282           fallbackMessage: 'Failed to persist cached API data.',
283           trackAnalytics: false,
284         });
285       });
286     }, PERSIST_DEBOUNCE_MS);
287   }
288
289   private async persistCache(): Promise<void> {
290     const serialized = Object.fromEntries(this.memoryCache.entries
          ());
291     await AsyncStorage.setItem(API_CACHE_STORAGE_KEY, JSON.
          stringify(serialized));
```

```
292   }
293
294   public async clearCache(): Promise<void> {
295     this.memoryCache.clear();
296     this.inFlightRequests.clear();
297     this.lastCachedDataUpdatedAt = null;
298     this.staleFallbackUsed = false;
299     await AsyncStorage.removeItem(API_CACHE_STORAGE_KEY);
300   }
301
302   public consumeStaleFallbackFlag(): boolean {
303     const staleFallbackUsed = this.staleFallbackUsed;
304     this.staleFallbackUsed = false;
305     return staleFallbackUsed;
306   }
307
308   public getLastCachedDataUpdatedAt(): number | null {
309     return this.lastCachedDataUpdatedAt;
310   }
311
312   private async fetchAPI<T>(endpoint: string): Promise<T> {
313     await this.ensureCacheHydrated();
314
315     const cacheKey = this.buildCacheKey(endpoint);
316     const cachedResponse = this.getCachedResponse<T>(cacheKey);
317     if (cachedResponse.data && !cachedResponse.isStale) {
318       this.lastCachedDataUpdatedAt = cachedResponse.updatedAt;
319       return cachedResponse.data;
320     }
321
322     const existingRequest = this.inFlightRequests.get(cacheKey);
323     if (existingRequest) {
324       return existingRequest as Promise<T>;
325     }
326
327     const requestPromise = (async () => {
328       try {
329         const url = `${this.baseURL}${endpoint}`;
330         const response = await fetch(url, {
331           method: 'GET',
332           headers: this.headers,
```

```
333         });
334
335         if (!response.ok) {
336           const errorData = await response.json().catch(() => null
                );
337           throw new Error(`HTTP error! status: ${response.status},
                 message: ${errorData?.status_message || 'Unknown
                error'}`);
338         }
339
340         const data = (await response.json()) as T;
341         this.setCachedResponse(cacheKey, data, this.getCacheTTL(
               endpoint));
342         this.lastCachedDataUpdatedAt = Date.now();
343         return data;
344       } catch (error) {
345         if (cachedResponse.data) {
346           this.staleFallbackUsed = true;
347           this.lastCachedDataUpdatedAt = cachedResponse.updatedAt;
348           return cachedResponse.data;
349         }
350
351         throw reportError('TMDBService.fetchAPI', error, {
352           fallbackMessage: 'Unable to load data from server.
                Please try again.',
353         });
354       } finally {
355         this.inFlightRequests.delete(cacheKey);
356       }
357     })();
358
359     this.inFlightRequests.set(cacheKey, requestPromise);
360
361     try {
362       return await requestPromise;
363     } catch (error) {
364       throw error;
365     }
366   }
367
368   public async getPopularTVShows(page = 1): Promise<APIResponse<
```

```
    TVShow>> {
369   return this.fetchAPI<APIResponse<TVShow>>(`/tv/popular?page=${
        page}`);
370 }
371
372 public async getTVShowDetails(id: number): Promise<TVShow> {
373   return this.fetchAPI<TVShow>(`/tv/${id}`);
374 }
375
376 public async getTVShowCast(id: number): Promise<CastMember[]> {
377   const response = await this.fetchAPI<TVShowCreditsResponse>(`/
        tv/${id}/credits`);
378   return Array.isArray(response.cast) ? response.cast : [];
379 }
380
381 public async searchTVShows(query: string, page = 1): Promise<
      APIResponse<TVShow>> {
382   return this.fetchAPI<APIResponse<TVShow>>(`/search/tv?query=${
        encodeURIComponent(query)}&page=${page}`);
383 }
384
385 public async getTopRatedTVShows(page = 1): Promise<APIResponse<
      TVShow>> {
386   return this.fetchAPI<APIResponse<TVShow>>(`/tv/top_rated?page=
        ${page}`);
387 }
388
389 public async getTVShowsAiringToday(page = 1): Promise<
      APIResponse<TVShow>> {
390   return this.fetchAPI<APIResponse<TVShow>>(`/tv/airing_today?
        page=${page}`);
391 }
392
393 public async getSeasonDetails(tvId: number, seasonNumber: number
      ): Promise<Season> {
394   return this.fetchAPI<Season>(`/tv/${tvId}/season/${
        seasonNumber}`);
395 }
396
397 public getImageUrl(path: string, size: string): string {
398   return `${TMDB_CONFIG.IMAGE_BASE_URL}/${size}${path}`;
```

```
  }
}

export default TMDBService.getInstance();
```

5.1.2 WatchlistService

WatchlistService manages the user's saved shows, watch progress, and watch history. It stores that data locally with AsyncStorage and can trigger cloud synchronization when Firebase is available. This service is important because it supports the app's personalized experience and connects the watchlist, notifications, and settings features.

```
import AsyncStorage from '@react-native-async-storage/async-
    storage';
import { TVShow, Episode } from './TMDBService';
import { reportError } from '@/app/utils/errorHandling';
import { hasEpisodeAired } from '@/app/utils/episodeRelease';

const WATCHLIST_STORAGE_KEY = '@EpisodeAlerts:watchlist';
const WATCH_PROGRESS_STORAGE_KEY = '@EpisodeAlerts:watchProgress';
const WATCH_HISTORY_STORAGE_KEY = '@EpisodeAlerts:watchHistory';

export interface LastWatchedEpisode {
  showId: number;
  showName: string;
  seasonNumber: number;
  episodeNumber: number;
  episodeName: string;
  watchedAt: number;
}

export interface WatchHistoryEntry extends LastWatchedEpisode {
  id: string;
}

export interface WatchlistEntry {
  show: TVShow;
  addedAt: number;
}

export interface WatchlistSnapshot {
  watchlist: WatchlistEntry[];
```

```
  progressMap: Record<string, LastWatchedEpisode>;
  historyMap: Record<string, WatchHistoryEntry[]>;
}

class WatchlistService {
  private static instance: WatchlistService;

  private constructor() {}

  public static getInstance(): WatchlistService {
    if (!WatchlistService.instance) {
      WatchlistService.instance = new WatchlistService();
    }
    return WatchlistService.instance;
  }

  private triggerCloudSync(): void {
    void import('./CloudSyncService')
      .then((module) => module.default.syncToCloudIfSignedIn())
      .catch((error) => {
        reportError('WatchlistService.triggerCloudSync', error, {
          fallbackMessage: 'Cloud sync could not be triggered.',
        });
      });
  }

  private normalizeWatchlist(rawWatchlist: unknown):
     WatchlistEntry[] {
    if (!Array.isArray(rawWatchlist)) {
      return [];
    }

    const now = Date.now();
    const normalized: WatchlistEntry[] = [];

    rawWatchlist.forEach((item, index) => {
      if (item && typeof item === 'object' && 'show' in item) {
        const entry = item as { show?: TVShow; addedAt?: number };
        if (!entry.show || typeof entry.show.id !== 'number') {
          return;
        }
```

```
70
71          normalized.push({
72            show: {
73              ...entry.show,
74              addedAt: entry.addedAt,
75            },
76            addedAt: typeof entry.addedAt === 'number' ? entry.
                addedAt : now,
77          });
78
79          return;
80        }
81
82        const show = item as TVShow;
83        if (!show || typeof show.id !== 'number') {
84          return;
85        }
86
87        const addedAt = show.addedAt ?? now - (rawWatchlist.length -
              index);
88        normalized.push({
89          show: {
90            ...show,
91            // Preserve original order for legacy arrays by creating
                  stable timestamps.
92            addedAt,
93          },
94          addedAt,
95        });
96      });
97
98      return normalized;
99    }
100
101   private async getWatchlistEntries(): Promise<WatchlistEntry[]> {
102     try {
103       const watchlistJson = await AsyncStorage.getItem(
            WATCHLIST_STORAGE_KEY);
104       const normalized = this.normalizeWatchlist(watchlistJson ?
            JSON.parse(watchlistJson) : []);
105       await this.saveWatchlistEntries(normalized);
```

```
106       return normalized;
107     } catch (error) {
108       reportError('WatchlistService.getWatchlistEntries', error, {
109         fallbackMessage: 'Unable to load watchlist entries.',
110       });
111       return [];
112     }
113   }
114
115   private async saveWatchlistEntries(entries: WatchlistEntry[]):
        Promise<void> {
116     await AsyncStorage.setItem(WATCHLIST_STORAGE_KEY, JSON.
          stringify(entries));
117   }
118
119   async getWatchlist(): Promise<TVShow[]> {
120     try {
121       const entries = await this.getWatchlistEntries();
122       return entries.map((entry) => ({
123         ...entry.show,
124         addedAt: entry.addedAt,
125       }));
126     } catch (error) {
127       reportError('WatchlistService.getWatchlist', error, {
128         fallbackMessage: 'Unable to load your watchlist.',
129       });
130       return [];
131     }
132   }
133
134   async getWatchlistSnapshot(): Promise<WatchlistSnapshot> {
135     const [watchlist, progressMap, historyMap] = await Promise.all
          ([
136       this.getWatchlistEntries(),
137       this.getProgressMap(),
138       this.getHistoryMap(),
139     ]);
140
141     return {
142       watchlist,
143       progressMap,
```

```
144       historyMap,
145     };
146   }
147
148   async restoreWatchlistSnapshot(
149     snapshot: WatchlistSnapshot,
150     options?: { skipCloudSync?: boolean },
151   ): Promise<boolean> {
152     try {
153       await this.saveWatchlistEntries(snapshot.watchlist);
154       await AsyncStorage.setItem(WATCH_PROGRESS_STORAGE_KEY, JSON.
            stringify(snapshot.progressMap));
155       await AsyncStorage.setItem(WATCH_HISTORY_STORAGE_KEY, JSON.
            stringify(snapshot.historyMap));
156
157       if (!options?.skipCloudSync) {
158         this.triggerCloudSync();
159       }
160
161       return true;
162     } catch (error) {
163       reportError('WatchlistService.restoreWatchlistSnapshot',
            error, {
164         fallbackMessage: 'Unable to restore watchlist snapshot.',
165       });
166       return false;
167     }
168   }
169
170   async addToWatchlist(show: TVShow): Promise<boolean> {
171     try {
172       const currentEntries = await this.getWatchlistEntries();
173
174       if (currentEntries.some((item) => item.show.id === show.id))
            {
175         return false;
176       }
177
178       const addedAt = Date.now();
179       const entry: WatchlistEntry = {
180         show: {
```

```
          ...show,
          addedAt,
        },
        addedAt,
      };

      const updatedWatchlist = [...currentEntries, entry];
      await this.saveWatchlistEntries(updatedWatchlist);
      this.triggerCloudSync();
      return true;
    } catch (error) {
      reportError('WatchlistService.addToWatchlist', error, {
        fallbackMessage: 'Unable to add this show to your
            watchlist.',
      });
      return false;
    }
  }

  async removeFromWatchlist(showId: number): Promise<boolean> {
    try {
      const currentWatchlist = await this.getWatchlistEntries();
      const updatedWatchlist = currentWatchlist.filter((show) =>
          show.show.id !== showId);

      await this.saveWatchlistEntries(updatedWatchlist);
      await this.clearLastWatchedEpisode(showId, { skipCloudSync:
          true });
      await this.clearWatchHistory(showId, { skipCloudSync: true
          });
      this.triggerCloudSync();
      return true;
    } catch (error) {
      reportError('WatchlistService.removeFromWatchlist', error, {
        fallbackMessage: 'Unable to remove this show from your
            watchlist.',
      });
      return false;
    }
  }

```

```
217   async isInWatchlist(showId: number): Promise<boolean> {
218     try {
219       const watchlist = await this.getWatchlistEntries();
220       return watchlist.some((show) => show.show.id === showId);
221     } catch (error) {
222       reportError('WatchlistService.isInWatchlist', error, {
223         fallbackMessage: 'Unable to check watchlist status.',
224         trackAnalytics: false,
225       });
226       return false;
227     }
228   }
229
230   async clearWatchlist(): Promise<boolean> {
231     try {
232       await AsyncStorage.removeItem(WATCHLIST_STORAGE_KEY);
233       await AsyncStorage.removeItem(WATCH_PROGRESS_STORAGE_KEY);
234       await AsyncStorage.removeItem(WATCH_HISTORY_STORAGE_KEY);
235       this.triggerCloudSync();
236       return true;
237     } catch (error) {
238       reportError('WatchlistService.clearWatchlist', error, {
239         fallbackMessage: 'Unable to clear your watchlist.',
240       });
241       return false;
242     }
243   }
244
245   private async getProgressMap(): Promise<Record<string,
       LastWatchedEpisode>> {
246     try {
247       const progressJson = await AsyncStorage.getItem(
          WATCH_PROGRESS_STORAGE_KEY);
248       return progressJson ? JSON.parse(progressJson) : {};
249     } catch (error) {
250       reportError('WatchlistService.getProgressMap', error, {
251         fallbackMessage: 'Unable to load watch progress.',
252       });
253       return {};
254     }
255   }
```

```
256
257   async getLastWatchedEpisodes(): Promise<Record<string,
          LastWatchedEpisode>> {
258     return this.getProgressMap();
259   }
260
261   private async getHistoryMap(): Promise<Record<string,
          WatchHistoryEntry[]>> {
262     try {
263       const historyJson = await AsyncStorage.getItem(
              WATCH_HISTORY_STORAGE_KEY);
264       return historyJson ? JSON.parse(historyJson) : {};
265     } catch (error) {
266       reportError('WatchlistService.getHistoryMap', error, {
267         fallbackMessage: 'Unable to load watch history.',
268       });
269       return {};
270     }
271   }
272
273   async getWatchHistory(showId: number): Promise<WatchHistoryEntry
          []> {
274     const historyMap = await this.getHistoryMap();
275     return (historyMap[String(showId)] || []).sort((a, b) => b.
          watchedAt - a.watchedAt);
276   }
277
278   private async appendWatchHistory(showId: number, showName:
          string, episode: Episode): Promise<void> {
279     const historyMap = await this.getHistoryMap();
280     const showKey = String(showId);
281     const existing = historyMap[showKey] || [];
282     const nextEntry: WatchHistoryEntry = {
283       id: `${showId}-${episode.season_number}-${episode.
              episode_number}-${Date.now()}`,
284       showId,
285       showName,
286       seasonNumber: episode.season_number,
287       episodeNumber: episode.episode_number,
288       episodeName: episode.name,
289       watchedAt: Date.now(),
```

```
290     };
291
292     const deduped = existing.filter(
293       (entry) =>
294         !(
295           entry.seasonNumber === episode.season_number &&
296           entry.episodeNumber === episode.episode_number &&
297           Math.abs(entry.watchedAt - nextEntry.watchedAt) < 60_000
298         ),
299     );
300
301     historyMap[showKey] = [nextEntry, ...deduped].slice(0, 50);
302     await AsyncStorage.setItem(WATCH_HISTORY_STORAGE_KEY, JSON.
          stringify(historyMap));
303   }
304
305   async getLastWatchedEpisode(showId: number): Promise<
          LastWatchedEpisode | null> {
306     const progressMap = await this.getProgressMap();
307     return progressMap[String(showId)] || null;
308   }
309
310   async setLastWatchedEpisode(showId: number, showName: string,
          episode: Episode): Promise<boolean> {
311     try {
312       if (!hasEpisodeAired(episode.air_date)) {
313         throw new Error('Cannot save progress for an unreleased
              episode.');
314       }
315
316       const progressMap = await this.getProgressMap();
317       const watchedAt = Date.now();
318       progressMap[String(showId)] = {
319         showId,
320         showName,
321         seasonNumber: episode.season_number,
322         episodeNumber: episode.episode_number,
323         episodeName: episode.name,
324         watchedAt,
325       };
326
```

```
327       await AsyncStorage.setItem(WATCH_PROGRESS_STORAGE_KEY, JSON.
              stringify(progressMap));
328       await this.appendWatchHistory(showId, showName, episode);
329       this.triggerCloudSync();
330       return true;
331     } catch (error) {
332       reportError('WatchlistService.setLastWatchedEpisode', error,
              {
333         fallbackMessage: 'Unable to save watch progress.',
334       });
335       return false;
336     }
337   }
338
339   async clearLastWatchedEpisode(showId: number, options?: {
          skipCloudSync?: boolean }): Promise<boolean> {
340     try {
341       const progressMap = await this.getProgressMap();
342       delete progressMap[String(showId)];
343       await AsyncStorage.setItem(WATCH_PROGRESS_STORAGE_KEY, JSON.
              stringify(progressMap));
344
345       if (!options?.skipCloudSync) {
346         this.triggerCloudSync();
347       }
348
349       return true;
350     } catch (error) {
351       reportError('WatchlistService.clearLastWatchedEpisode',
              error, {
352         fallbackMessage: 'Unable to clear watch progress.',
353       });
354       return false;
355     }
356   }
357
358   async clearWatchHistory(showId: number, options?: {
          skipCloudSync?: boolean }): Promise<boolean> {
359     try {
360       const historyMap = await this.getHistoryMap();
361       delete historyMap[String(showId)];
```

```
362       await AsyncStorage.setItem(WATCH_HISTORY_STORAGE_KEY, JSON.
              stringify(historyMap));
363
364       if (!options?.skipCloudSync) {
365         this.triggerCloudSync();
366       }
367
368       return true;
369     } catch (error) {
370       reportError('WatchlistService.clearWatchHistory', error, {
371         fallbackMessage: 'Unable to clear watch history.',
372       });
373       return false;
374     }
375   }
376 }
377
378 export default WatchlistService.getInstance();
```

Chapter 6

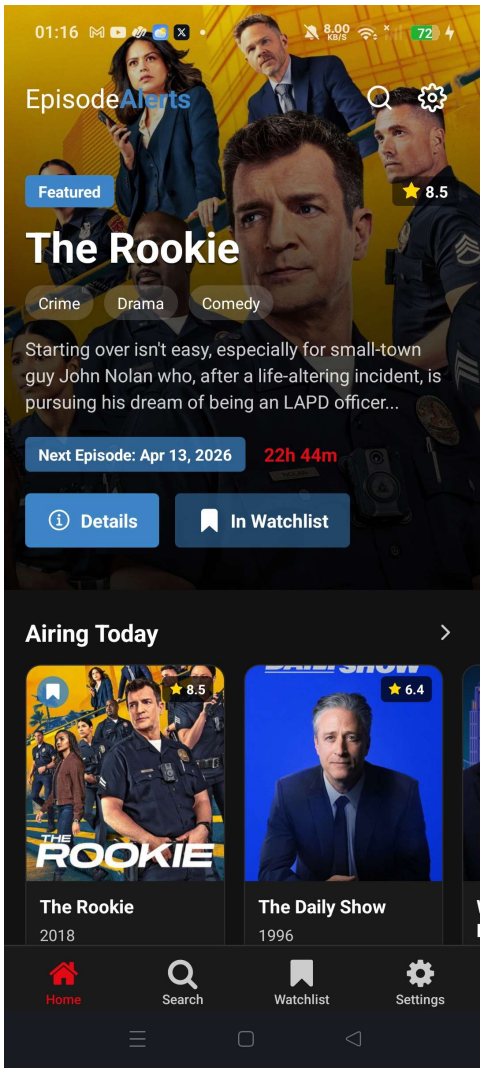
Results

6.1 Results Overview

The screenshots below show the implemented app in its final form, covering the main user flows: discovery, show details, watchlist management, analytics, and settings. Together, they demonstrate that the core screens are working as intended and that the app presents a consistent experience across its main features.

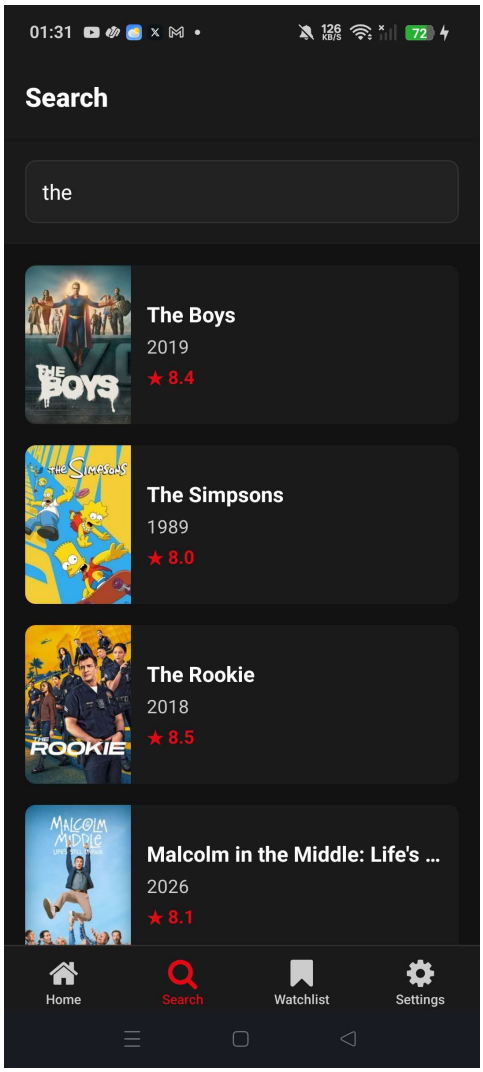
Source code is available on GitHub at EpisodeAlerts-MobileApp.

6.2 Application Screenshots

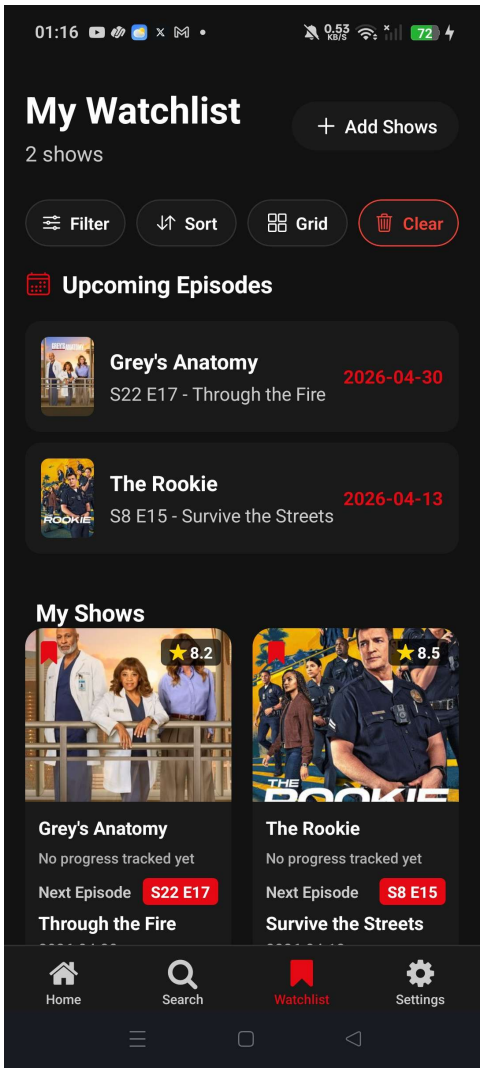


Home screen with featured content and category sections.

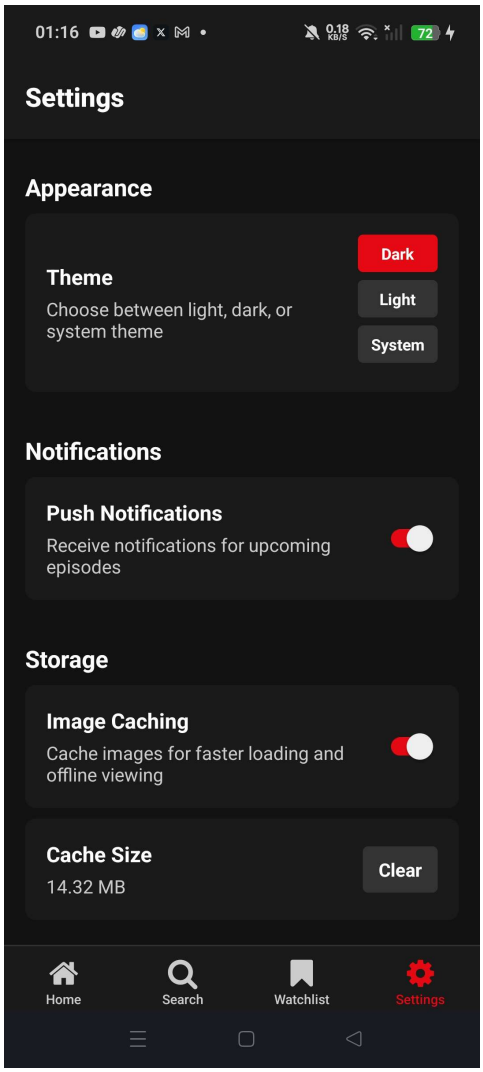
Show details screen with episode information and watchlist state.



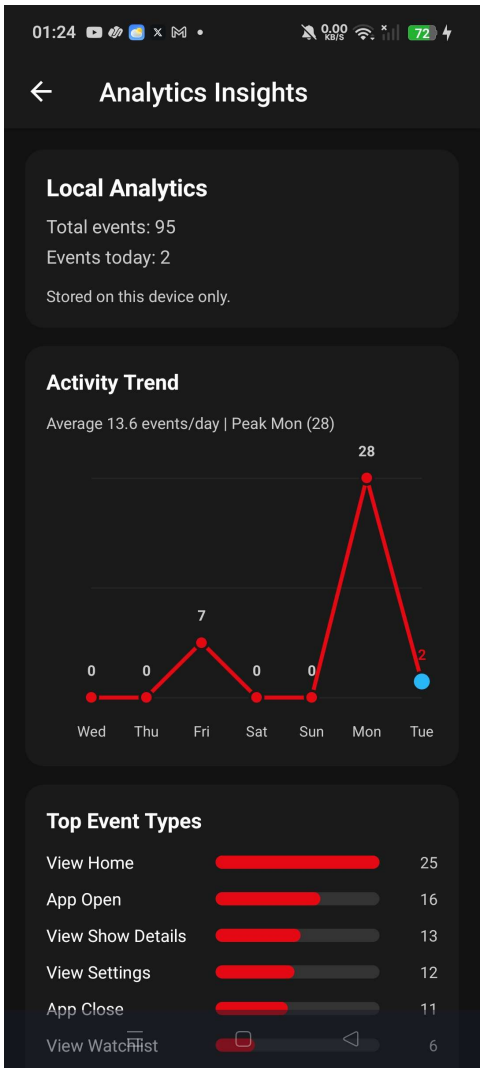
Search screen showing TV show search results.



Watchlist screen with upcoming episodes and saved shows.



Settings screen with theme, notifications, and caching options.



Analytics insights screen showing usage trends and event breakdowns.

6.3 Outcome Summary

The implemented build satisfies the core functional objectives defined in the SRS and demonstrates stable behavior across the primary user journeys.

- Discovery and search workflows successfully fetch and display TV data from TMDB, while preserving usability through cache and fallback behavior.
- Show details, season navigation, and next-episode context are presented consistently from Home, Search, and Watchlist entry points.
- Watchlist actions (add, remove, and update) persist correctly in local storage and reflect immediately in the user interface.
- Settings changes for theme, analytics, and notifications are retained and applied across app sessions.
- The analytics screen confirms event tracking visibility, and optional cloud-sync be-

havior aligns with the configured authentication state.

6.4 Conclusion

Episode Alerts delivers a complete and coherent mobile experience for TV discovery and tracking, with a clear architecture and reliable end-to-end flows. The final implementation is suitable for submission and provides a strong baseline for future enhancements such as richer personalization, expanded recommendation logic, and broader sync capabilities.

References

Documentation

[1] **The Movie Database (TMDB) API documentation.**
TMDB docs. Accessed 16 Apr. 2026.

[2] **Expo documentation.**
Expo docs. Accessed 16 Apr. 2026.

[3] **React Native documentation.**
React Native docs. Accessed 16 Apr. 2026.

[4] **Firebase documentation.**
Firebase docs. Accessed 16 Apr. 2026.

[5] **Episode Alerts project repository.**
Project repository. Accessed 16 Apr. 2026.

Related Work and Context

[1] **CineSync Tracker.**
App Store listing for a movie and TV watchlist app with discovery and release-tracking features.
App Store listing. Accessed 16 Apr. 2026.

[2] **Expo post discussing CineSync Tracker and TMDB usage.**
Original post by the app author describing the app and its Expo-based implementation.
Original post. Accessed 16 Apr. 2026.

[3] **Follow-up Expo comment thread.**
Discussion of TMDB as the content source and the free-tier rate limit.
Comment thread. Accessed 16 Apr. 2026.